UNITED STATES INTERNATIONAL UNIVERSITY – AFRICA

School of Science and Technology

Department of Computing

System Integration Report

Smart Health Symptom Checker and Doctor Recommendation System

Submitted by

Amy Wanjugu Njau – 669008

Course: SWE3090XA – Software Engineering Final Project

Supervisor: Mr Fredrick Ogore

Date of submission: June 2026

Summer Semester 2026

Table of Contents

1. Introduction

1.1 Purpose of the Integration

This report describes the integration of the components that make up the Smart Health Symptom Checker and Doctor Recommendation System. Rather than integrating separate enterprise products, the system integrates several internal subsystems and one external service into a single, coherent workflow: a mobile client, a back-end API hosting a rule-based diagnostic engine, a cloud database, a geolocation service, and the external Google Maps Platform. The purpose of the integration is to streamline the user's journey from symptom entry to actionable guidance, ensure consistent data flow between layers, and present a unified experience despite the underlying components being independent.

1.2 Scope of Systems Involved

The integration covers five subsystems: the Flutter mobile client (presentation), the Node.js/Express REST API (application logic), the rule-based diagnostic engine (reasoning), Firebase Firestore (data storage), and the Google Maps Platform together with the device geolocation service (external location data). The scope is limited to the interfaces and data exchanged between these components; it does not extend to third-party telemedicine, electronic health records, or payment systems.

1.3 Objectives Achieved Through Integration

- A single end-to-end workflow connecting symptom input, diagnosis, specialist recommendation, and nearby-provider lookup.
- Consistent, well-typed data exchange between the client, server, and database through a defined API contract.
- Reliable use of external location data while isolating the rest of the system from the specifics of the Maps provider.
- Graceful degradation so that the failure of one component (for example, denied location access) does not break the whole experience.

2. System Overview

Each integrated subsystem and its role is described below.

Subsystem	Key Functionality	Dependencies / Constraints
Flutter Mobile Client	Captures symptoms, displays ranked conditions and specialist advice, shows nearby providers on a map/list.	Depends on the REST API and device location permission; constrained by mobile connectivity.
Node.js / Express API	Exposes REST endpoints, orchestrates diagnosis and provider lookup, returns structured JSON.	Stateless; depends on the engine, database, and Maps Platform availability.
Rule-Based Diagnostic Engine	Scores and ranks candidate conditions from a weighted symptom matrix; maps to specialists.	Depends on the knowledge base; accuracy bounded by rule coverage.
Firebase Firestore	Stores the knowledge base, specialist mappings, provider records, and query logs.	Cloud-hosted; subject to read/write quotas and network access.
Google Maps Platform + Geolocation	Provides device location and nearby healthcare facilities.	External API; rate limits and cost; requires user permission.

Table 2.1: Integrated subsystems.

3. Integration Architecture

3.1 Approach

The integration follows an API-based, layered approach rather than point-to-point coupling. The mobile client never talks to the database or the Maps Platform directly; instead, all interactions pass through the Express REST API, which acts as a single integration point and mediator. This keeps the subsystems loosely coupled — each can change internally as long as the API contract is honoured — and gives the architecture a microservice-leaning character where the diagnostic engine and provider service are independent, replaceable modules behind the API.

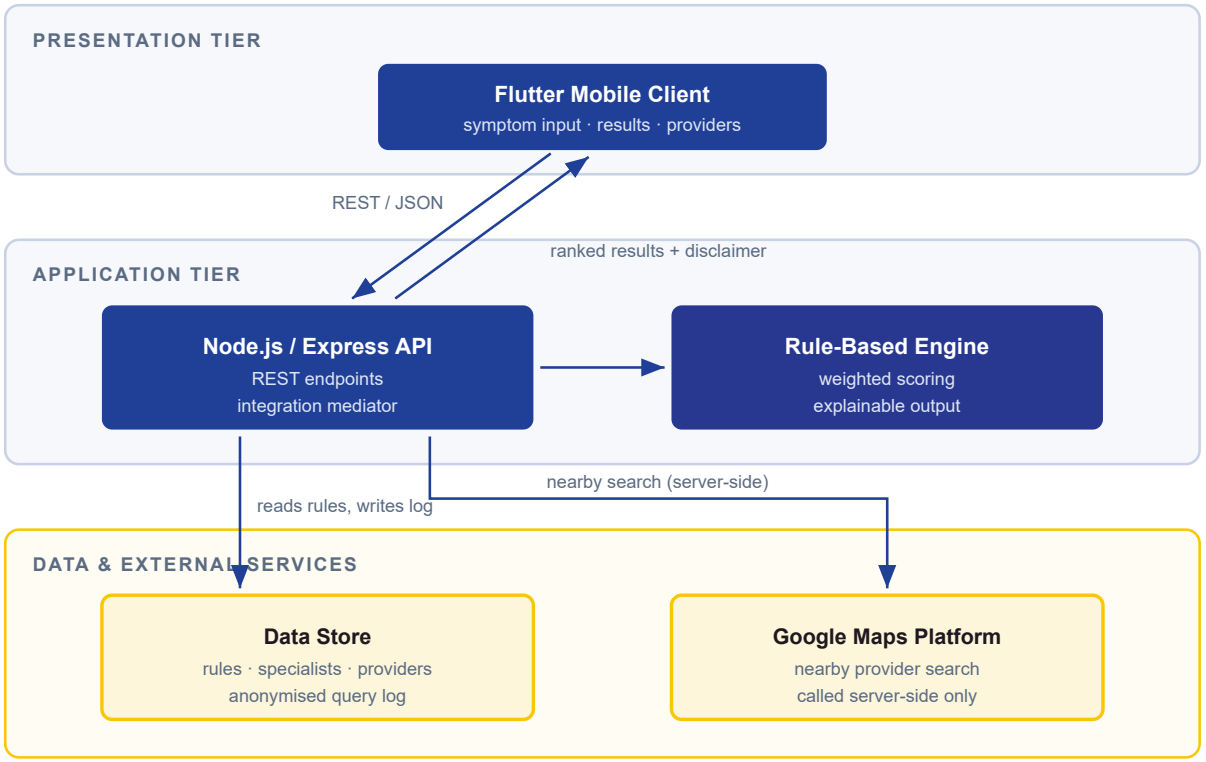

Figure 3.1: Integration architecture.

3.2 Data Flow

Information moves through the system in a defined sequence: the client collects symptoms and sends them to the API; the API invokes the diagnostic engine, which reads rules from Firestore and returns ranked conditions and a recommended specialist; the client then sends the user's coordinates and the specialist type back to the API, which queries the Maps Platform and returns ranked nearby providers. Each query and result can be logged to Firestore for analytics.

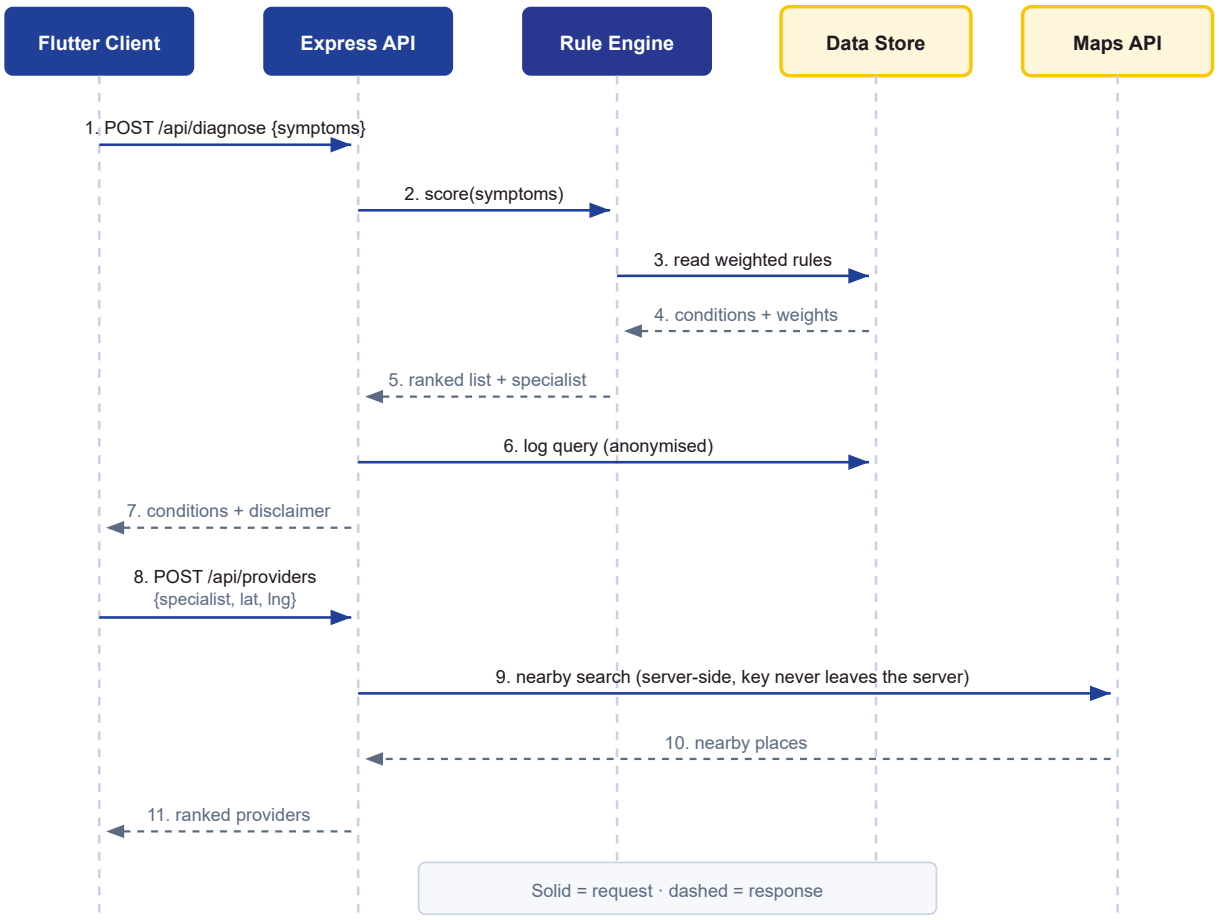

Figure 3.2: End-to-end data flow.

3.3 Protocols

Communication between the client and the API uses REST over HTTPS, with JSON as the data-interchange format. Calls to the Google Maps Platform are made server-side over HTTPS using its REST endpoints. The design favours simple, stateless request/response exchanges; message queues are not required at the current scale but could be introduced later for asynchronous logging or batch enrichment.

3.4 Security

Security spans authentication, encryption, and access control. All traffic travels over HTTPS, so data is encrypted in transit. The Maps Platform key is held server-side in environment variables and never exposed to the client, preventing key leakage. Incoming requests are validated before processing, and access to the location subsystem is gated by explicit user permission. Persisted user data is minimised, and token-based authentication and encrypted storage are planned to protect any future accounts.

4. Implementation Details

4.1 Steps Taken During Integration

- Defined the API contract (endpoints, request/response schemas) as the integration boundary between client and server.
- Implemented the diagnosis endpoint to invoke the engine and return ranked conditions, the specialist, and a disclaimer.
- Implemented the provider endpoint to call the Google Maps Platform with the user's coordinates and specialist type.
- Built a client service layer that calls the endpoints and maps JSON responses into typed models for the UI.
- Configured environment variables and Firestore access, then connected the engine to the knowledge base.
- Tested each interface individually and then end-to-end across the integrated workflow.

4.2 Tools and Technologies

- **Flutter / Dart** — mobile client and HTTP service layer.
- **Node.js / Express** — REST API and integration mediator.
- **Firebase Firestore** — cloud database for rules, mappings, providers, and logs.
- **Google Maps Platform** — geolocation and nearby-place data.
- **Git / GitHub** — version control; **Postman** — API testing.

4.3 Representative Integration Code

The client service below illustrates how the front-end integrates with the back-end through the diagnosis endpoint.

`lib/services/api_service.dart` (excerpt)

```
Future<DiagnosisResult> getDiagnosis(List<String> symptoms) async {
  final response = await http.post(
    Uri.parse('$baseUrl/api/diagnose'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'symptoms': symptoms}),
  );
  if (response.statusCode == 200) {
    return DiagnosisResult.fromJson(jsonDecode(response.body));
  } else {
    throw Exception('Diagnosis request failed: ${response.statusCode}');
  }
}
```

4.4 Challenges and Solutions

Challenge	Solution Applied
Google Maps API cost and rate limits	Performed lookups server-side, cached results, and limited query frequency.
Handling denied location permission	Caught the denial on the client and continued the flow with symptom and specialist guidance only.
Keeping client and server data models in sync	Defined a single JSON contract and typed model classes mapped directly to it.
Protecting the Maps API key	Moved all Maps calls server-side and stored the key in environment variables.

Table 4.1: Integration challenges and solutions.

5. Testing & Validation

5.1 Test Cases Executed

ID	Test Case	Expected Result	Status
T1	Submit valid symptoms to `/diagnose`	Ranked conditions + specialist returned in JSON	Pass
T2	Submit empty symptom list	400 error with a clear validation message	Pass
T3	Request providers with valid coordinates	Ranked nearby providers returned	Pass
T4	Deny location permission	Flow continues with symptom/specialist guidance	Pass
T5	Simulate network failure on client	Readable error with retry option, no crash	Pass
T6	End-to-end: symptoms → conditions → providers	Complete workflow without data loss	Pass

Table 5.1: Integration test cases.

5.2 Validation Methods

Validation combined three levels of testing. Unit tests checked the diagnostic engine in isolation — confirming that scores are computed and ranked correctly for known symptom sets. Integration tests exercised the API together with the engine, database, and Maps Platform to confirm the components work as a whole. User-acceptance testing then verified the end-to-end experience from the user's perspective, focusing on clarity, correctness of recommendations, and graceful handling of errors.

5.3 Results Summary and Issue Resolution

The integrated workflow performed as expected across the core scenarios, with data passing correctly between layers and failures handled gracefully. Early issues — chiefly around external API limits and client/server model mismatches — were resolved through caching, request throttling, and a single shared data contract. Remaining work is the addition of an automated regression suite so that future changes to the knowledge base or endpoints are validated continuously.

6. Outcomes & Benefits

6.1 Improved Efficiency

Integrating the subsystems into one workflow removes manual steps for the user: a single sequence takes them from symptoms to a probable condition, the right specialist, and a reachable provider, where previously each would require separate, disconnected effort.

6.2 Enhanced Data Consistency and Accuracy

Routing all interactions through the API and a shared JSON contract means the client, engine, and database operate on consistent, well-typed data. Centralising the knowledge base in Firestore ensures every diagnosis draws on the same source of truth.

6.3 Better User Experience

The integration is invisible to the user, which is the goal: they experience one smooth application rather than a set of disconnected services. Visual feedback, clear error handling, and the safety disclaimer combine to make the experience trustworthy and easy to follow.

7. Recommendations

7.1 Improvements for Scalability

- Run the stateless API behind a load balancer with multiple instances as demand grows.
- Introduce a caching layer and, if needed, a message queue for asynchronous logging and provider enrichment.

7.2 Monitoring and Maintenance

- Add centralised logging, health-check endpoints, and alerting on API errors and Maps quota usage.
- Establish an automated test and deployment pipeline so integrations are validated on every change.

7.3 Future Integration Opportunities

- Integrate optional telemedicine or appointment-booking services once the core is stable.
- Add multilingual content and offline support to extend reach in low-connectivity settings.
- Validate and enrich the knowledge base against recognised medical references and datasets.

8. Conclusion

8.1 Summary of Integration Success

The integration successfully unifies the mobile client, REST API, diagnostic engine, database, and external location services into a single, coherent system. The API-based, layered approach keeps the subsystems loosely coupled and resilient, and testing confirmed that data flows correctly and failures are handled gracefully across the integrated workflow.

8.2 Final Remarks on Sustainability and Adaptability

The chosen architecture is sustainable and adaptable: because integration happens through a single, well-defined API boundary, new subsystems can be added and existing ones replaced with minimal disruption. Acting on the recommendations — scalability, monitoring, and automated validation — would carry the integration from a working prototype to a robust, production-ready platform.

Appendix A: Entity Relationship Diagram

The data exchanged across the integration is organised around the entities below, and is shown as the formal ER diagram in Figure A.1.

Entity	Key Fields	Relationship
User	`user_id` (PK), name, location, created_at	Initiates many Queries (1–M).
Symptom	`symptom_id` (PK), name, category	Linked to Conditions (M–M).
Condition	`condition_id` (PK), name, description, severity	Linked to Symptoms (M–M); maps to one Specialist.
Condition_Symptom	`condition_id` (FK), `symptom_id` (FK), weight	Associative entity holding the rule weight.
Specialist	`specialist_id` (PK), type, description	Recommended by many Conditions.
Provider	`provider_id` (PK), name, specialty, latitude, longitude, rating	Offers one or more Specialist types (M–M).
Query	`query_id` (PK), `user_id` (FK), symptoms, result, timestamp	Belongs to one User.

Table A.1: Core entities exchanged across the integration.

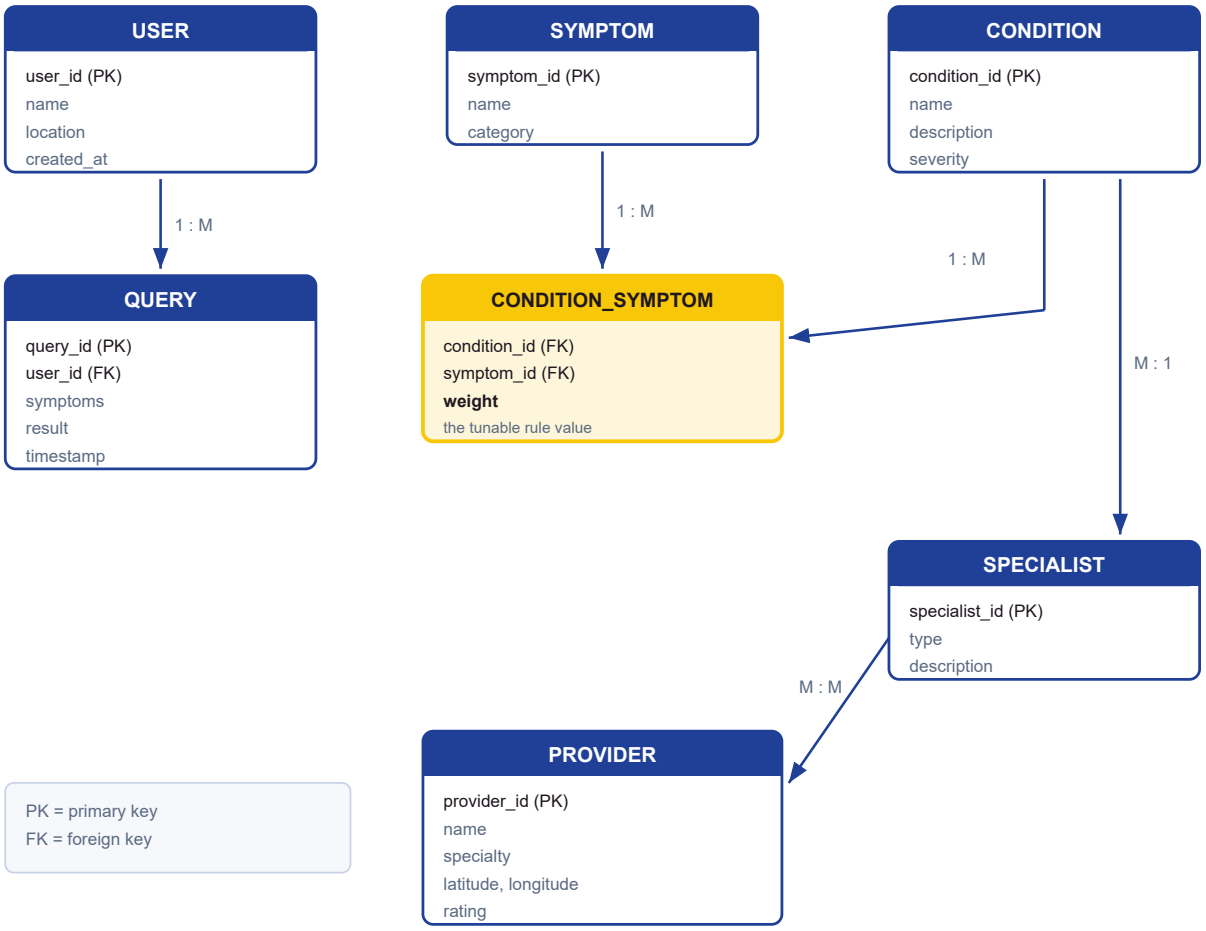

Figure A.1: Entity relationship diagram.